

Project - High Level Design

on

Autonomous Logistics Researcher Agent

Course Name: Agentic AI

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AKSHAT BAJAJ	EN22CS301088
2	AKANKASHA PARIHAR	EN22CS301079
3	AJAY PAL SINGH THAKUR	EN22CS301077
4	AKSHITA FARKYA	EN22CS301099
5	AASHITA CHAVHAN	EN22CS301018
6	AAKRATI JAIN	EN22CS301013

Group Name: Group 05D1

Project Number: AAI-05

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-26

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

The Autonomous Logistics Researcher Agent is an advanced AI-driven system designed to automate research activities in the logistics domain. The system is built using modern Agentic AI concepts, where intelligent agents independently perform tasks such as searching, analyzing, and summarizing information.

Unlike traditional research systems that require manual effort and predefined workflows, this system dynamically adapts to user queries. It breaks down complex problems into smaller tasks, performs multi-step reasoning, and gathers information from multiple sources.

The integration of Large Language Models (LLMs), along with frameworks like LangChain and CrewAI, allows the system to generate meaningful, context-aware summaries. These summaries are stored in a knowledge repository, enabling reuse and continuous learning.

The system significantly reduces manual effort, improves accuracy, and provides structured insights for logistics-related research.

1.1 Scope of the Document

This document provides a detailed view of the internal design of the system, focusing on how different components are structured and interact.

Functional Scope

- Accept natural language queries from users
- Perform automated research using intelligent agents
- Extract and filter relevant logistics information
- Generate structured summaries using LLM
- Store outputs for future reference

Technical Scope

- Module-level design
- API-level interactions
- Data flow and processing logic
- Agent communication mechanism

Operational Scope

- Error handling strategies
- Performance considerations
- Security aspects

Out of Scope

- Detailed frontend UI design
- Infrastructure-level deployment (e.g., Kubernetes)
- Real-time streaming systems

1.2 Intended Audience

Developers

Helps in understanding system architecture, module responsibilities, and implementation logic.

Test Engineers

Provides insight into workflows, enabling validation of system behavior and handling of edge cases.

Project Evaluators

Allows evaluation of system complexity, modularity, and design approach.

DevOps Engineers

Helps in understanding system dependencies and preparing deployment strategies.

1.3 System Overview

The system functions as an intelligent research pipeline that automates the process of collecting, analyzing, and summarizing logistics-related information.

Working Overview

The system begins by accepting a user query. It then processes the query to understand its intent and context. Based on this understanding, it divides the task into smaller sub-tasks, which are handled by different agents.

The research agent collects data from various web sources. This data is filtered and refined before being passed to the summarization agent, which generates a structured output. Finally, the output is stored and presented to the user.

Key Capabilities

- Autonomous decision-making by agents
- Multi-source data aggregation
- Context-aware summarization
- Knowledge storage and reuse

Example Scenario

For a query like "Future of AI in logistics," the system:

- Identifies keywords such as AI, logistics, and trends
- Searches multiple sources
- Extracts relevant insights
- Generates a structured summary

2. System Design

Architecture Approach

The system follows a hybrid approach combining:

- Agent-Based Architecture for intelligent task execution
- Layered Architecture for structured organization

Layer-wise Structure

Presentation Layer:

Handles user interaction by accepting queries and displaying results.

Agent Layer:

Manages multiple agents and controls execution flow using CrewAI.

Processing Layer:

Implements core logic using LangChain and LLM APIs for data processing.

Tool Layer:

Integrates external services such as web search APIs for data collection.

Storage Layer:

Stores processed outputs in a structured format for future access.

Design Approach

The design ensures separation of concerns, allowing each layer to perform a specific function without dependency conflicts.

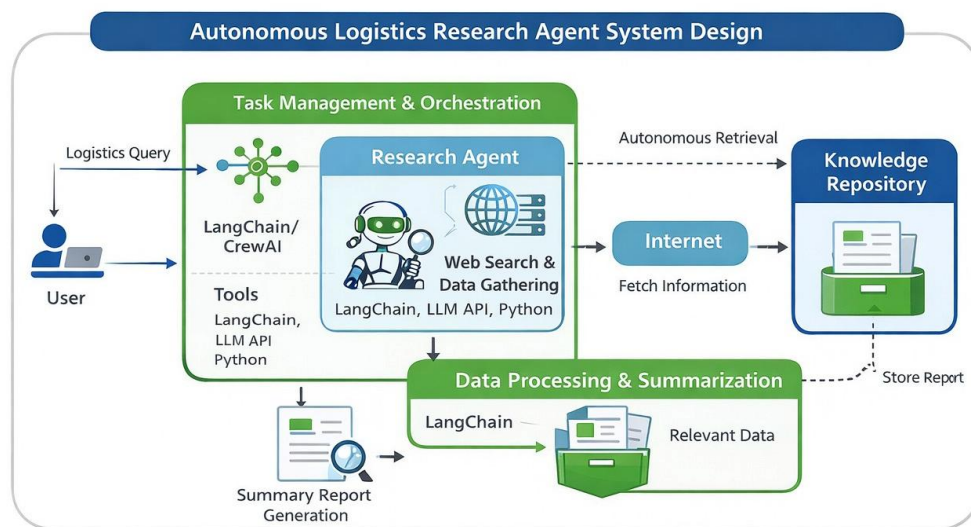


Fig: System design

2.1 Application Design

Application design explains how modules are structured and interact with each other.

Query Processing Module

This module acts as the entry point of the system. It validates user input, removes unnecessary data, and extracts meaningful keywords to improve processing accuracy.

Agent Orchestrator Module

This module controls the workflow of the system. It assigns tasks to agents, maintains execution order, and ensures proper coordination between modules.

Research Module

Responsible for collecting data from external sources. It performs web searches, gathers information from multiple sources, and filters out irrelevant or duplicate data.

Summarization Module

This module processes the collected data using LLM and generates structured summaries. It ensures that the output is accurate, relevant, and easy to understand.

Storage Module

Handles saving the generated summaries. It maintains a repository of previous queries and results, enabling future reuse.

2.2 Process Flow

The process flow describes how the system executes tasks step by step.

Execution Steps

1. User submits a query
2. Query is validated and processed
3. Task is assigned to agents
4. Research agent collects data
5. Data is filtered and refined
6. Summarization agent generates output
7. Output is stored in repository
8. Final result is displayed

Error Handling

The system includes mechanisms to handle failures:

- Retry logic for API failures
- Handling empty responses
- Logging errors for debugging



Fig: Process flow

2.3 Information Flow

Information flow explains how data moves through the system.

Data Flow Stages

- Input: User query
- Intermediate: Raw web data
- Processed: Filtered insights
- Output: Final structured summary

Data Transformation

Data is transformed in multiple stages:

- Raw data is cleaned
- Clean data is filtered
- Filtered data is structured into summary

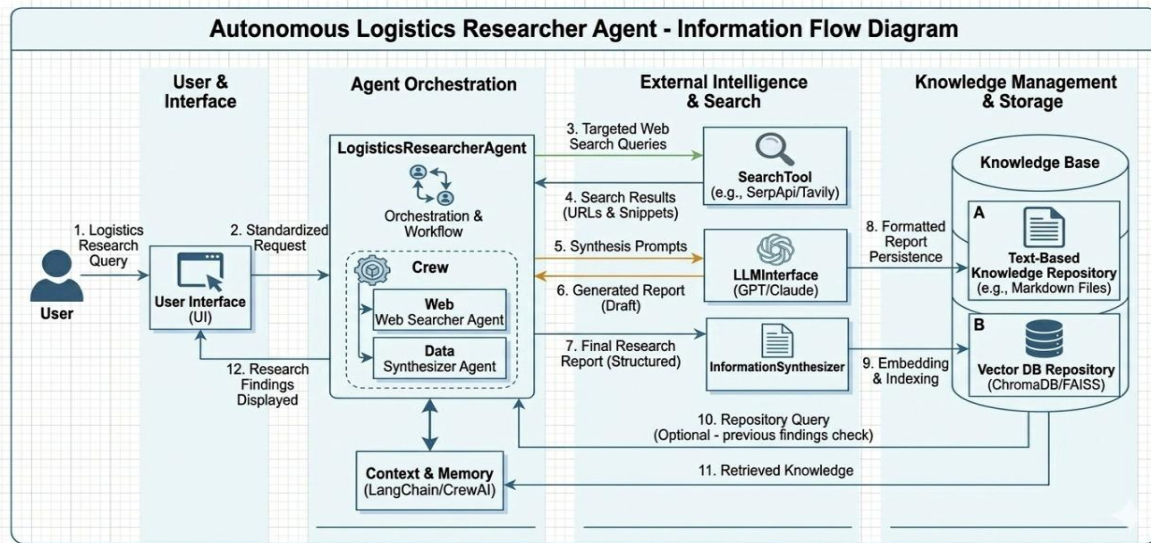


Fig: Information Flow diagram

2.4 Components Design

Component design describes the individual parts of the system.

Main Components

Query Handler:

Processes and validates user input.

Agent Orchestrator:

Controls workflow and coordinates agents.

Research Agent:

Collects and filters data from web sources.

Summarization Agent:

Generates structured output using LLM.

Storage Agent:

Stores results and maintains repository.

Key Methods

- processQuery()
- searchWeb()
- summarize()
- saveToFile()

2.5 Key Design Considerations

Scalability

The system can be expanded by adding new agents or tools without affecting existing components.

Performance

Efficient processing ensures quick response time even for complex queries.

Security

API keys and sensitive data are securely handled to prevent unauthorized access.

Reliability

The system is designed to handle failures gracefully and provide consistent results.

Maintainability

The modular design makes it easy to update, debug, and enhance the system.

2.5 API Catalogue

Overview

The API catalogue defines structured communication between system components and external services.

Query Processing API

Endpoint: /api/v1/query

Method: POST

Description:

Processes user queries and returns summarized output.

Request:

```
{  
  "query": "Latest logistics trends"  
}
```

Response:

```
{
  "status": "success",
  "data": {
    "summary": "Key trends include automation, AI integration, and smart logistics."
  }
}
```

Knowledge Storage API

Endpoint: /api/v1/knowledge/save**Method:** POST**Description:**

Stores generated summaries.

Request:

```
{
  "query": "AI in logistics",

  "summary": "AI improves efficiency and automation."
}
```

Response:

```
{
  "status": "success",
  "message": "Stored successfully"
}
```

External APIs

LLM API:

Used for generating summaries and understanding queries.

Web Search API:

Used for retrieving real-time information from the internet.

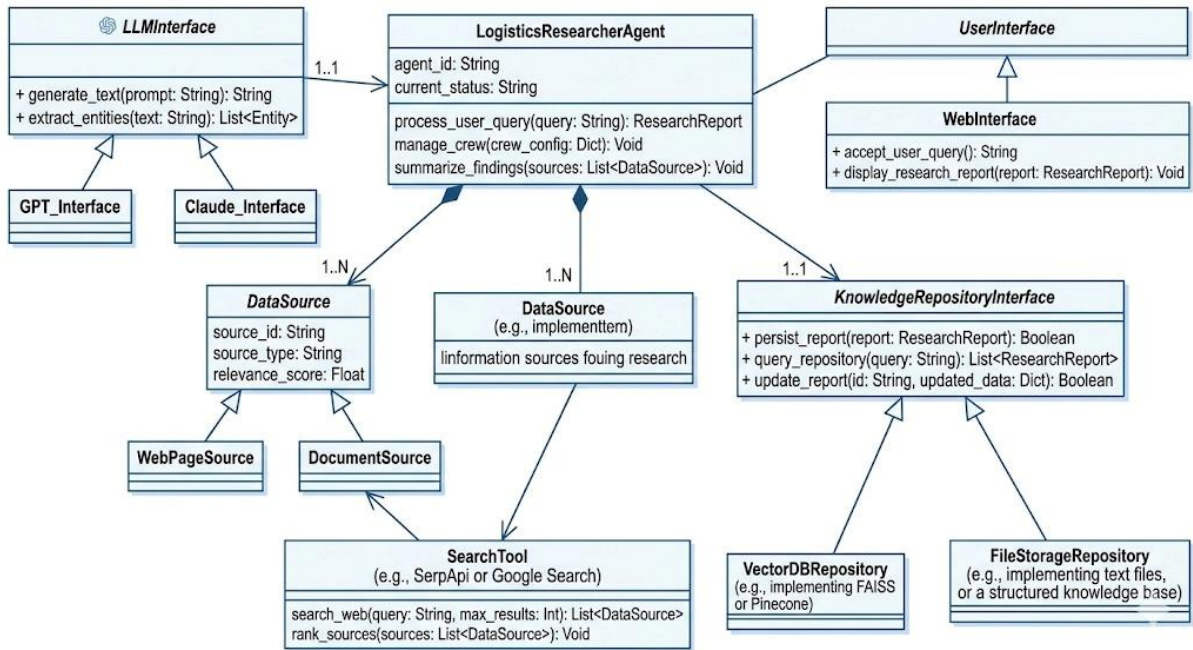


Fig: Class Diagram

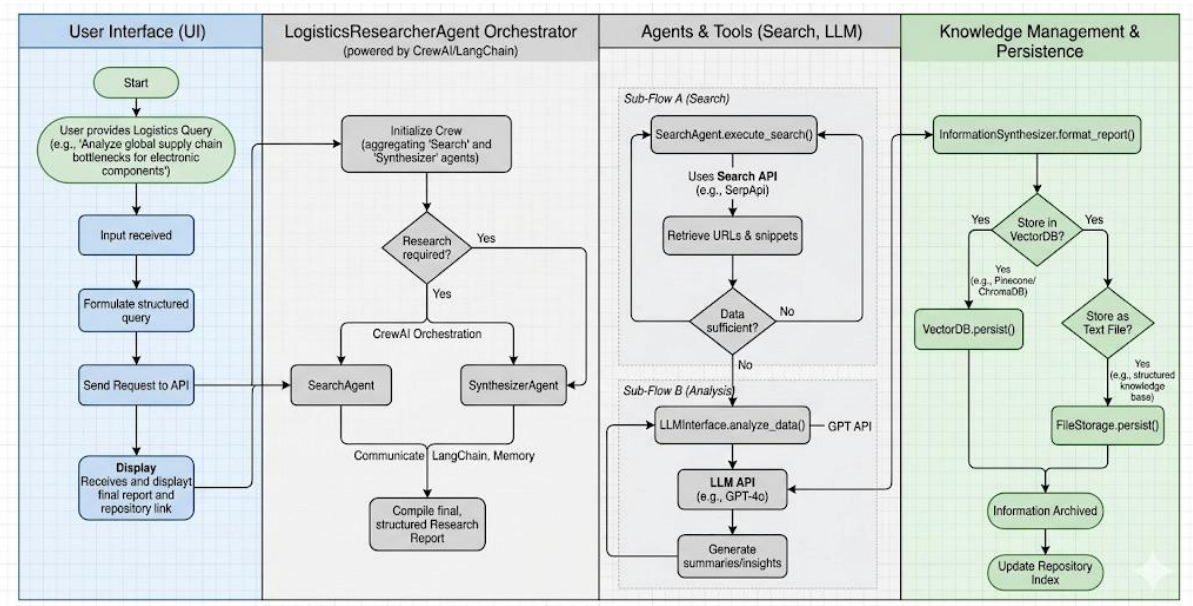


Fig: User Flow diagram

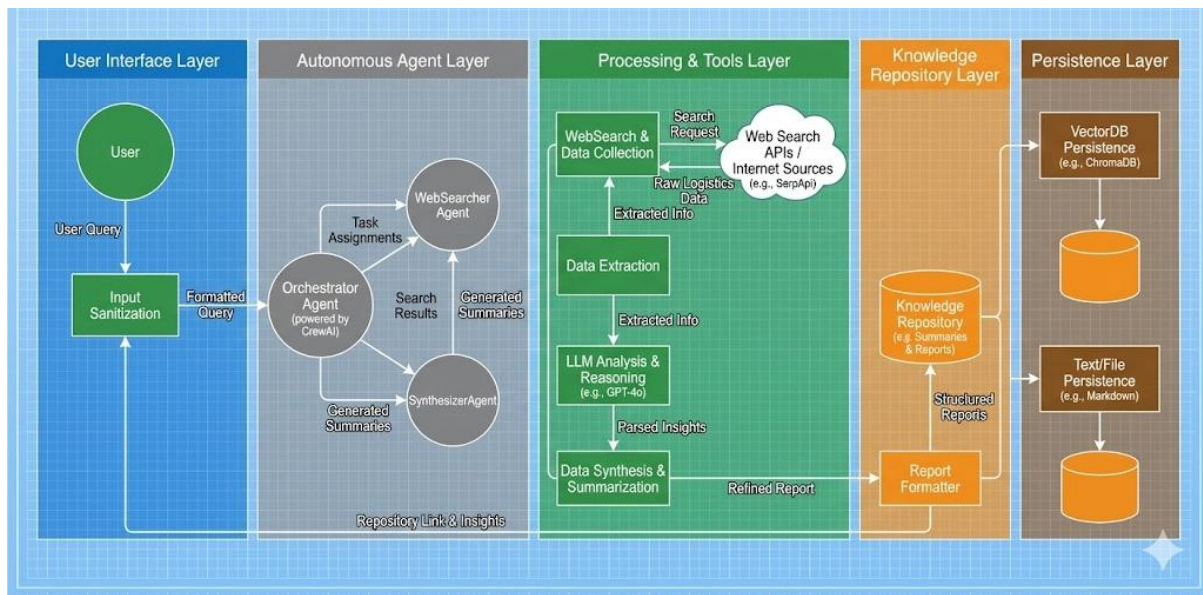


Fig: Data flow diagram

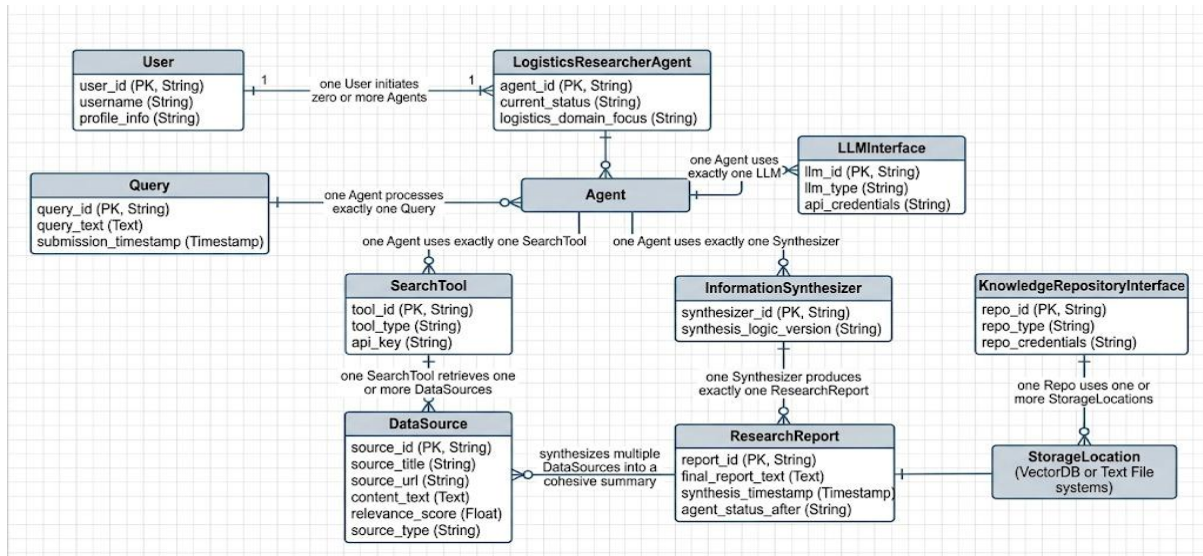


Fig: ER diagram

3. Data Design

3.1 Data Model (High-Level)

The system follows a structured yet flexible data model to efficiently store user queries, processed summaries, and source information. The design ensures easy retrieval, scalability, and reusability of knowledge.

Entities and Attributes

1. Query Entity

Stores user input queries submitted to the system.

- query_id: Unique identifier for each query
- text: Original user query
- created_at: Timestamp of query submission

2. Result Entity

Stores the summarized output generated by the system.

- result_id: Unique identifier
- query_id: Reference to the associated query
- summary: Generated structured summary
- timestamp: Time of generation

3. Source Entity

Stores information about data sources used during research.

- source_id: Unique identifier
- query_id: Associated query reference
- url: Source link
- content: Extracted relevant content

Relationships

- One Query → Multiple Sources
- One Query → One Result
- Result is derived from multiple Sources

3.2 Data Access Mechanism

The system uses API-driven data access to ensure modularity and scalability.

- Data is accessed through RESTful APIs for both read and write operations
- Indexed storage is used to improve query performance and reduce retrieval time
- Frequently accessed queries are optimized using caching mechanisms
- Data validation is performed before storage to ensure consistency

3.3 Data Retention Policies

To maintain efficiency and manage storage effectively, the system follows defined data retention strategies:

- Frequently accessed queries and their results are retained for quick reuse
- Older or less frequently used data is archived periodically
- Duplicate or redundant entries are removed to maintain data quality
- Logs and temporary data are cleared after a fixed duration

3.4 Data Migration

The system supports data migration to ensure flexibility and reliability.

- Data can be exported and imported in structured formats (e.g., JSON)
- Backup strategies are implemented to prevent data loss
- Migration scripts can be used to transfer data between storage systems
- Versioning ensures compatibility during system upgrades

4. Interfaces

4.1 Internal Interfaces

Internal interfaces define communication between system components.

- **Agent ↔ Orchestrator:**
The orchestrator assigns tasks to agents and manages execution flow
- **LLM ↔ Processing Layer:**
The processing layer sends structured prompts to the LLM and receives summarized outputs
- **Modules Interaction:**
Query processing, research, summarization, and storage modules communicate via defined APIs

4.2 External Interfaces

External interfaces enable integration with third-party systems.

- **Web APIs:**
Used to fetch real-time logistics data from online sources
- **LLM APIs:**
Used for natural language understanding and summary generation
- **Optional Integrations:**
Can include databases, analytics tools, or monitoring services

5. State and Session Management

The system is primarily designed to be stateless for better scalability and performance.

- Each API request is independent and does not rely on previous states
- Session tracking can be implemented if user-specific history is required
- Temporary session data can be stored using caching mechanisms
- Stateless design ensures easier scaling and load balancing

6. Caching

Caching is implemented to improve performance and reduce redundant computations.

- Frequently asked queries and their results are cached
- Reduces repeated calls to external APIs and LLM services
- Improves response time for similar queries
- Cache can be implemented using in-memory stores like Redis
- Cache invalidation strategies ensure updated results when needed

7. Non-Functional Requirements

7.1 Security

Security is a critical aspect of the system design.

- API keys and sensitive credentials are securely stored and encrypted
- Access control mechanisms restrict unauthorized usage
- Secure communication protocols (HTTPS) are used

- Input validation prevents injection and malicious attacks

7.2 Performance

The system is optimized to deliver fast and efficient results.

- Low latency response through optimized workflows
- Parallel execution of agents where applicable
- Efficient summarization using optimized LLM prompts
- Reduced API calls through caching and batching
- Scalable architecture to handle increasing load

8. References

- LangChain Documentation
- CrewAI Documentation
- OpenAI / LLM API Docs
- Logistics Industry Reports